

Bomb Lab

汇编分析与调试讲义

bomb406 · 整理于 2026 年 10 月 5 日

1. 实验目标与资料

通过反汇编与 GDB 观察，恢复六个关卡的输入条件，再从成功处理函数中找到隐藏关入口。每一关都遵循“读取输入 → 检查条件 → 成功处理”的流程；失败调用 `explode_bomb`，成功后调用 `phase_defused` 与课程服务器校验。

本讲义依据此前远端调试聊天的 69 轮记录、本次 phase 4–6 与隐藏关记录，并核对本地 **bomb406** 的二进制、**bomb.c** 和 **bomblab.pdf**。历史中的临时输入与错误推测已改为最终推导。所有地址和答案均针对 **bomb406**，不能直接套用到另一颗 bomb。

文件	用途
<code>bomb</code>	Linux x86-64 可执行程序，包含真正的关卡逻辑。
<code>bomb.c</code>	主程序：逐行读取输入，依次调用六关与成功处理函数。
<code>bomb.asm</code>	通过 <code>objdump -d bomb</code> 导出的汇编，方便静态阅读。
<code>.gdbinit</code>	GDB 启动配置：答案文件、断点与历史中的安全化脚本。
<code>psol.txt</code>	答案文件；每个非空输入行对应一次 <code>read_line</code> 。
<code>bomblab.pdf</code>	课程实验说明，包括环境、自动上报和隐藏关说明。

实际运行应在课程指定的 xLab Linux 容器中进行。macOS 上可以阅读文件，但这里的 Linux 程序及初始化检查不适合直接在本机运行。

1.1. 启动与输入

```
cd ~/ics-2026/bomb406
chmod u+x ./bomb
objdump -d ./bomb > bomb.asm
gdb ./bomb
```

`chmod u+x` 恢复当前用户的执行权限；解压过程可能丢失这一权限。`gdb ./bomb` 本身只是加载程序，但本次 `.gdbinit` 末尾有 `run`，所以会自动开始执行。

```
set args psol.txt
break phase_1
break phase_2
break phase_3
break phase_4
break phase_5
break phase_6
break phase_defused
```

`set args psol.txt` 等价于让被调试程序以 `./bomb psol.txt` 的参数运行。答案文件读到末尾后，程序会转而等待终端输入；所以“没有出现 (gdb) 提示符”可能只是正在等下一行。修改答案文件后必须保存，再用 `run` 从头验证；当前已经读入的字符串不会随文件修改而自动变化。

2. 阅读汇编与 GDB 的必要知识

2.1. 参数、返回值与比较

Linux x86-64 的整数或指针参数通常依次放入 `%rdi`、`%rsi`、`%rdx`、`%rcx`、`%r8`、`%r9`；整数返回值在 `%eax`。`%rbx`、`%rbp`、`%r12` 等被调用者保存寄存器常用来跨递归调用保存数据。

AT&T 语法一般是“源，目标”。`cmp a,b` 根据 $b - a$ 设置条件码，不写回结果；`test x,x` 用来检查零值与符号。`je/jne` 检查相等与不等；`jg/jge/jle` 使用有符号比较，`ja/jbe` 使用无符号比较。`lea` 只做地址或算术计算，不读取内存。

```
cmp $0x1,%eax      # Test eax against 1.
jle failure        # Branch if eax <= 1.
lea (%rax,%rbx),%r12d # r12d = eax + ebx.
```

`$ax` 只有 16 位，`$eax` 是 32 位，`$rax` 是 64 位。查看本实验的 `int` 返回值应使用 `p/d $eax`，不要用 `$ax` 代替。

2.2. 常用命令

命令	作用
<code>disas phase_4</code>	查看指定函数；分页时按 <code>c</code> 显示完。
<code>b *(phase_4+0x45)</code>	在函数起点加字节偏移的位置设断点。
<code>x/i \$pc</code>	确认即将执行的指令。断点停在该指令执行前。
<code>ni/si</code>	执行一条指令；前者跨过函数调用，后者进入函数。
<code>run/c</code>	重新开始 / 从当前停点继续。
<code>p/d \$eax</code>	以十进制打印寄存器或表达式的值。
<code>x/s \$rsi</code>	把指针指向的内存读作 C 字符串。
<code>x/6dw \$rsp</code>	读取六个 4 字节有符号十进制整数。
<code>x/gx &node1+0</code>	按 8 字节十六进制显示地址处的数据。
<code>bt/frame 1</code>	查看调用栈 / 切换到调用者栈帧。
<code>info breakpoints</code>	确认断点位置、编号、是否启用以及脚本。
<code>disable N/enable N</code>	禁用 / 启用编号为 <code>N</code> 的断点。
<code>layout asm/layout regs</code>	打开汇编 / 寄存器视图。

退出布局：先按 `Ctrl+x`，再按 `a`；刷新画面：`Ctrl+l`。断点编号由创建顺序决定，重启或新增断点后可能不同，应先查询，不能把旧记录中的编号当作固定编号。

2.3. 地址与栈位置

静态地址如 `0x16ab`，加载后可能变成 `0x5555555556ab`。用 函数名+偏移 设断点可避免依赖加载基址；偏移单位是字节，必须对应指令起点。第一关实际调用比较函数的位置为 `phase_1+11`，教程中的 `+15` 会落进指令内部。

栈偏移只在确定的程序位置有效。`push`、`sub %rsp` 和 `call` 都会改变栈位置，进入被调用函数后，不能继续把调用者的 `$rsp+8` 当作同一输入。先用 `x/i $pc` 与 `bt` 确定停点；需要读调用者局部变量时，切换到相应栈帧。

2.4. sscanf：格式、输出地址与返回值

`sscanf` 从已有字符串中按格式解析数据。它的返回值是成功赋值的项目数，不是读到的某个整数。对于下面的调用，前两个参数是字符串和格式，后两个参数是写入结果的地址：

```
int a, b;
int count = sscanf(input, "%d %d", &a, &b);
```

在第四关调用前，寄存器与栈的关系为：

```

RDI = input          first argument
RSI = "%d %d"        second argument
RDX = RSP + 8 = &a   third argument
RCX = RSP + 12 = &b   fourth argument

```

```

RSP + 8 ... RSP + 11   a: 4 bytes
RSP + 12 ... RSP + 15  b: 4 bytes

```

`lea 0x8(%rsp),%rdx` 传的是地址；`mov 0x8(%rsp),%eax` 读的是该地址保存的整数。这两类指令的区别直接决定能否正确恢复参数。

```

lea 0xc(%rsp),%rcx    # Set the address for the second output.
lea 0x8(%rsp),%rdx    # Set the address for the first output.
lea ...(%rip),%rsi    # Set the format string address.
mov $0,%eax           # No vector arguments in this variadic call.
call __isoc99_sscanf
cmp $2,%eax           # Require two assigned outputs.
jne failure

```

调用前清零 `%eax` 是可变参数调用约定的一部分，用于表示没有通过向量寄存器传递的参数；它不是输入值初始化。调用返回后 `%eax` 已变为成功转换数量。输入 24 2 返回 2，输入 24 x 返回 1；没有读到的输出位置可能仍是旧内容，因此不能只看栈里是否“恰好有答案”。

六个整数的读取需要八个函数参数：输入、格式、六个输出指针。前六个参数使用寄存器，最后两个参数放在栈中，所以 `read_six_numbers` 中会出现额外的 `push`。

```

mov %rsi,%rdx         # Output 0: a.
lea 0x4(%rsi),%rcx    # Output 1: a + 1.
lea 0x14(%rsi),%rax
push %rax             # Output 5: stack argument 8.
lea 0x10(%rsi),%rax
push %rax             # Output 4: stack argument 7.
lea 0xc(%rsi),%r9     # Output 3: register argument 6.
lea 0x8(%rsi),%r8     # Output 2: register argument 5.
lea ...(%rip),%rsi    # Replace RSI with the format address.
call __isoc99_sscanf

```

这里 `%rsi` 起初是数组首地址，最后才被替换为格式地址。恢复参数时必须按指令执行顺序追踪同一寄存器的角色变化。

2.5. 将汇编整理为高级语言的步骤

先标出所有失败调用与返回点，再区分入口检查、循环体、循环测试和更新指令。给寄存器临时命名为 `i`、`sum`、`state` 等语义变量，最后写等价 C 代码。以下 C 片段用于表达已恢复的检查关系，不是原始源码；省略寄存器保存、栈对齐和输出提示。示例中的 `explode_bomb()` 按终止程序处理。

3. 历史中的安全化与成功校验

本次安全化只修改远端 `.gdbinit` 的调试行为，没有修改 `bomb` 二进制。实际核对得到：`explode_bomb` 起点为 `0x1c89`，失败上报调用为 `0x1cb7`，一个退出调用为 `0x1cd9`，对应偏移为 `0x2e` 与 `0x50`。

```

# Historical configuration for bomb406 only.
break *(explode_bomb + 0x2e)
commands
  silent
  jump *(explode_bomb + 0x50)
end

```

这段历史配置在失败消息发送前转到退出调用。**BOOM!!!** 的打印发生在此断点之前，所以仍可能出现；它不代表失败消息一定已经发送。跳转也跳过了退出参数设置，因而退出状态码不能证明成功。它仅在 GDB 加载该配置且断点启用时生效，直接运行 `./bomb` 不会读取 `.gdbinit`。

先前未经验证的 `phase_defused+0x2a` 跳转已被禁用，成功处理仍执行服务器校验。调试时可另设 `break explode_bomb`，在函数入口暂停并用 `bt` 定位失败分支；此时应先检查，不要继续执行失败路径。

本地检查通过、停在 `phase_defused`、服务器接受答案，是三个不同的阶段。`phase_defused` 先将状态置 0，再调用 `send_msg(1, 指针)`；返回后只有状态为 1 才继续，否则退出。六关共用这一流程。

4. Phase 1: 字符串比较

第一关将用户字符串与固定字符串传给 `strings_not_equal`，要求返回值为 0。

```
1575: lea  ...(%rip),%rsi
157c: call strings_not_equal
1581: test %eax,%eax
1583: jne  failure
```

调用前 `%rdi` 指向输入，`%rsi` 指向期望字符串。实际断点偏移是 `0x157c-0x1571=11`。

```
disas phase_1
b *(phase_1+11)
c
x/s $rdi
x/s $rsi
```

将读出的字符串原样保存为第一行，不加引号，保留大小写、空格和末尾句点：

Love is a program that science and logic can never explain.

4.1. 比较函数为什么返回 0

`strings_not_equal` 先比较长度，长度不同返回 1；长度相同后逐字节比较，任何字符不同返回 1，直到结束符都一致才返回 0。可恢复为：

```
/** Return 0 for equal strings, or 1 for different strings. */
int strings_not_equal(const char *a, const char *b) {
    if (string_length(a) != string_length(b)) {
        return 1;
    }
    while (*a != '\0') {
        if (*a != *b) {
            return 1;
        }
        ++a;
        ++b;
    }
    return 0;
}
```

调用者的 `test %eax,%eax` 将返回值与零作逻辑测试；`jne` 在结果非零时进入失败路径。因此这里“0”代表相等，与有些返回布尔值的检查函数含义相反。

```

/** Check the exact phase 1 input string. */
void phase_1(const char *input) {
    const char *expected =
        "Love is a program that science and logic can never explain.";
    if (strings_not_equal(input, expected) != 0) {
        explode_bomb();
    }
}

```

5. Phase 2: 六个整数与循环

read_six_numbers 把六个 int 依次写入栈中：地址为 `$rsp+4i`。在 `phase_2+13`，读取已经完成，第一项检查还未执行。

```

b *(phase_2+13)
c
x/6dw $rsp

```

前两项固定为 0、1；循环从 $i = 2$ 到 $i = 5$ 检查后四项。下标乘 4，因为每个整数占 4 字节。

```

mov (%rsp,%rax,4),%eax # Read a[i-1].
add (%rsp,%rcx,4),%eax # Add a[i-2].
cmp %eax, (%rsp,%rdx,4) # Compare the sum with a[i].

```

递推关系为 $a_0 = 0, a_1 = 1, a_i = a_{i-1} + a_{i-2}$ ，因此第二关答案为：

```
0 1 1 2 3 5
```

5.1. 完整循环：寄存器与下标对应

这里 `%ebx=i`, `%rdx=i`, `%rcx=i-2`, `%rax` 起初是 $i-1$ ，读数组后又变成求和结果。一个寄存器可以在同一轮循环内先作为下标、再作为数值。

```

15b0: mov $2,%ebx
15b5: jmp 15ba
15b7: add $1,%ebx
15ba: cmp $5,%ebx
15bd: jg 15df # End when i > 5.
15bf: movslq %ebx,%rdx # rdx = i.
15c2: lea -2(%rbx),%ecx
15c5: movslq %ecx,%rcx # rcx = i - 2.
15c8: lea -1(%rbx),%eax
15cb: cltq # rax = i - 1.
15cd: mov (%rsp,%rax,4),%eax # eax = a[i-1].
15d0: add (%rsp,%rcx,4),%eax # eax += a[i-2].
15d3: cmp %eax, (%rsp,%rdx,4)
15d6: je 15b7 # Continue on equality.
15d8: call explode_bomb

```

`movslq` 把 32 位整数符号扩展为 64 位，`cltq` 专门把 `%eax` 符号扩展到 `%rax`。这些指令为地址计算准备 64 位下标；它们没有改变这里合法的小整数数值。

```

/** Check six integers against the required recurrence. */
void phase_2(const char *input) {
    int a[6];
    read_six_numbers(input, a);
    if (a[0] != 0 || a[1] != 1) {
        explode_bomb();
    }
    for (int i = 2; i < 6; ++i) {
        if (a[i] != a[i - 1] + a[i - 2]) {
            explode_bomb();
        }
    }
}

```

第一次跳到 15ba 检查 $i=2$ ；每轮成功后跳到 15b7 执行 $i++$ 。不是先加 1 再检查第 2 项，所以没有遗漏 $a[2]$ 。依次推导为 $a[2]=1$ 、 $a[3]=2$ 、 $a[4]=3$ 、 $a[5]=5$ 。

6. Phase 3: 跳转表与共享算术路径

这一关读两个整数。sscanf 的第三个参数 $\%rdx$ 指向 $\$rsp+12$ ，第四个参数 $\%rcx$ 指向 $\$rsp+8$ ，所以第一个数在 $+12$ ，第二个数在 $+8$ 。不能沿用第四关的输入布局。

第一个数先经无符号范围检查，要求在 0-7，再作为跳转表下标。表项是**有符号 32 位相对位移**：读取后符号扩展，加上表基址，再间接跳转。每项占 4 字节，不是每项 8 字节的绝对地址表。最终还有 第一个数 ≤ 5 的检查，所以实际可选下标为 0-5。

本次选下标 0，对应路径先令 $\%eax=239$ ，随后执行共享算术链：

$$239 - 609 + 712 - 300 + 300 - 300 + 300 - 300 = 42。$$

最终用 `cmp %eax,0x8(%rsp)` 检查第二个数是否等于计算结果。

```

b *(phase_3+26)
# Continue to the sscanf call, then inspect its arguments.
c
x/s $rsi
ni
p/d $eax
x/2dw $rsp+8
b *(phase_3+121)
c
p/d $eax

```

sscanf 返回 2 表示成功转换两个整数；最后断点处 $\%eax=42$ 。一组有效答案为：

```
0 42
```

6.1. 间接跳转：位移怎样变为地址

```

1609: cml $7,0xc(%rsp)
160e: ja failure
1614: mov 0xc(%rsp),%eax
1618: lea ...(%rip),%rdx      # Table base: static address 0x3300.
161f: movslq (%rdx,%rax,4),%rax
1623: add %rdx,%rax          # Target = base + signed offset.
1626: jmp *%rax

```

`jmp *%rax` 的星号表示目标地址来自寄存器，区别于直接跳到固定标签。第 0 项的原始 4 字节为 6e e3 ff ff：按小端序组成 0xfffffe36e，作为有符号整数是 -7314。于是静态目标为 $0x3300 - 7314 = 0x166e$ ，正好跳到 `mov $0xef,%eax`，即初始化为 239。

负的输入下标以无符号解释会很大，所以第一处 `ja` 同时排除负数。跳转表允许 0-7，只说明这些下标有对应代码；随后 `cmp $5` 与 `jg` 还会排除 6、7。需要追到共同的出口，不能只读入口的范围检查。

6.2. 算术链与所有可用下标

```
166e: mov $0xef,%eax      # Case 0 starts with 239.
1673: jmp 1634
1634: sub $0x261,%eax      # Subtract 609.
1639: add $0x2c8,%eax      # Add 712.
163e: sub $0x12c,%eax      # Subtract 300.
1643: add $0x12c,%eax
1648: sub $0x12c,%eax
164d: add $0x12c,%eax
1652: sub $0x12c,%eax
1657: cml $5,0xc(%rsp)
165c: jg failure
165e: cmp %eax,0x8(%rsp)
1662: je success
```

其他下标从算术链的不同位置进入，共享后续指令。逐个追踪可得下表；选择 0 只是其中一组答案。

第一个数	第二个数	计算起点与结果
0	42	239-609+712-300=42
1	-197	0-609+712-300=-197
2	412	0+712-300=412
3	-300	0-300+300-300+300-300=-300
4	0	0+300-300+300-300=0
5	-300	0-300+300-300=-300

```
/** Check the selected phase 3 path and its result. */
void phase_3(const char *input) {
    int index, value;
    if (sscanf(input, "%d %d", &index, &value) != 2) {
        explode_bomb();
    }
    if ((unsigned)index > 7u) {
        explode_bomb();
    }
    const int results[8] = {42, -197, 412, -300, 0, -300, 0, -300};
    int expected = results[index];
    if (index > 5 || value != expected) {
        explode_bomb();
    }
}
```

这里用结果数组表达各路径的最终值，不表示原程序真的保存了这张结果表；原程序保存的是代码目标的相对位移。

7. Phase 4: 带参数的递归

7.1. 输入布局与范围

格式为 `%d %d`，第一个数 a 存在 `$rsp+8`，第二个数 b 存在 `$rsp+12`。两次比较要求 $1 < b \leq 4$ ，即 b 可取 2、3、4。随后调用 `func4(5,b)`，要求返回值等于 a 。

```

mov 0xc(%rsp),%esi
mov $0x5,%edi
call func4
cmp %eax,0x8(%rsp)

```

7.2. 从汇编恢复递推

%ebp 保存原始 n , %ebx 保存 b , %r12d 保存第一次递归结果加 b 。lea (%rax,%rbx,1),%r12d 是加法，不是把递归结果乘 2。

$$F(n, b) = \begin{cases} 0 & \text{若 } n \leq 0 \\ b & \text{若 } n = 1 \\ F(n-1, b) + b + F(n-2, b) & \text{若 } n \geq 2 \end{cases}$$

n	0	1	2	3	4	5
$F(n, b)$	0	b	$2b$	$4b$	$7b$	$12b$

所以 $a = 12b$, 有效输入为 24 2、36 3 或 48 4。本次使用:

24 2

7.3. 两次递归之间保存了什么

第一次调用会覆盖 %eax, 第二次调用又会覆盖它, 所以第一次的返回值必须先移到能跨调用保留的寄存器中。原始 n 与参数 b 也必须保留, 用于准备第二次递归。

```

16b3: mov %edi,%ebp          # Save the original n.
16b5: mov %esi,%ebx          # Save b.
16b7: cmp $1,%edi
16ba: je 16e0                # Return b for n == 1.
16bc: lea -1(%rdi),%edi      # First argument: n - 1.
16bf: call func4
16c4: lea (%rax,%rbx),%r12d  # Save F(n-1,b) + b.
16c8: lea -2(%rbp),%edi      # Use the saved original n.
16cb: mov %ebx,%esi          # Restore b for the second call.
16cd: call func4
16d2: add %r12d,%eax          # Add the saved first result.

```

lea -2(%rbp),%edi 使用原始 n , 不是在已经减过 1 的 %edi 上再减 2。递归调用前后的 push/pop 成对保存并恢复 %r12/%rbp/%rbx, 所以内层递归不会破坏外层保存的值。

```

/** Return F(n,b) for the small integers used by phase 4. */
int func4(int n, int b) {
    if (n <= 0) {
        return 0;
    }
    if (n == 1) {
        return b;
    }
    int first = func4(n - 1, b) + b;
    int second = func4(n - 2, b);
    return first + second;
}

```

以 $b = 2$ 为例: $F(2, 2) = 2 + 2 + 0 = 4$, $F(3, 2) = 4 + 2 + 2 = 8$, $F(4, 2) = 8 + 2 + 4 = 14$, $F(5, 2) = 14 + 2 + 8 = 24$ 。每一级都额外加一次 b , 所以它不是普通的斐波那契数列。

```

/** Check two integers against func4(5,b). */
void phase_4(const char *input) {
    int a, b;
    if (sscanf(input, "%d %d", &a, &b) != 2) {
        explode_bomb();
    }
    if (b <= 1 || b > 4) {
        explode_bomb();
    }
    if (a != func4(5, b)) {
        explode_bomb();
    }
}

```

入口第一条 `test %edi,%edi` 已经处理 $n \leq 0$ 。该分支直接返回，根本没有执行后面的压栈指令，因此也不需要弹栈； $n = 1$ 的分支则经过保存寄存器，必须跳回共同恢复位置再返回。

7.4. 动态验证与历史问题

```

b *(phase_4+26)
c
x/s $rsi
x/s $rdi
ni
p/d $eax
x/2dw $rsp+8
b *(phase_4+69)
c
p/d $eax
x/2dw $rsp+8

```

在 `sscanf` 执行后，返回值应为 2，栈中应为 24 2；在最终比较前，`%eax` 应为 24。历史里输入 0 2 时已实际观察到递归返回 24。

若读数异常，先确认 `$pc`。历史中的 `$ax=1`、`$rsp+16=24` 来自未确认停点与不同偏移，不能据此否定递推。若停在 `explode_bomb` 入口，`call` 已经把返回地址压入栈；应先 `bt`，再 `frame 1` 查看第四关的输入。

解析失败、范围失败共用 `phase_4+50` 的调用位置；仅靠这个位置不能区分二者。栈上也可能留有旧数据，应同时检查 `sscanf` 返回值与实际输入字符串。参考题中的 5 20 属于另一版本，其递推为 $2f(n-1) + f(n-2)$ ，并累加五次，不能套用。

8. Phase 5：字符低四位与查表求和

输入长度必须为 6。每个字符编码与 `0xf` 做按位与，得到下标 0-15；用该下标读取一个 4 字节表项，六项之和必须等于 `0x27=39`。

```

movzbl (%rbx,%rdx,1),%edx
and $0xf,%edx
add (%rsi,%rdx,4),%ecx

```

读取表时用 `x/16dw 0x555555557320`；这是本次调试的加载地址，重启后应以反汇编显示的当前地址为准。本次得到：

下标	0	1	2	3	4	5	6	7
值	2	10	6	1	12	16	9	3
下标	8	9	10	11	12	13	14	15
值	4	7	14	5	11	8	15	13

选择 a (编码 0x61, 下标 1, 值 10)、c (下标 3, 值 1)、h (下标 8, 值 4)。

aaachh

长度为 6, 和为 $3 \times 10 + 1 + 2 \times 4 = 39$ 。课程说明要求第五关答案只用数字和字母; 这一选择满足要求。

8.1. 循环控制与等价代码

```
1749: mov $0,%ecx          # sum = 0.
174e: mov $0,%eax            # i = 0.
1753: jmp 1773
175c: movslq %eax,%rdx        # Extend i for address calculation.
175f: movzbl (%rbx,%rdx),%edx # Read input[i] as an unsigned byte.
1763: and $0xf,%edx           # Keep only the low four bits.
1766: lea ...(%rip),%rsi       # Load the table address.
176d: add (%rsi,%rdx,4),%ecx   # sum += table[index].
1770: add $1,%eax             # i++.
1773: cmp $5,%eax
1776: jle 175c                # Process indices 0 through 5.
1778: cmp $0x27,%ecx
177b: jne failure
```

%eax 是循环下标, %ecx 是累加器, %edx 先是下标的副本, 随后被读出的字符覆盖, 最后变为表下标。movzbl 表示把 1 字节零扩展为 32 位, 避免高位旧值影响计算。

```
/** Check six characters by their low-bit table sum. */
void phase_5(const char *input) {
    const int table[16] = {
        2, 10, 6, 1, 12, 16, 9, 3,
        4, 7, 14, 5, 11, 8, 15, 13
    };
    if (string_length(input) != 6) {
        explode_bomb();
    }
    int sum = 0;
    for (int i = 0; i < 6; ++i) {
        unsigned index = (unsigned char)input[i] & 0x0fu;
        sum += table[index];
    }
    if (sum != 39) {
        explode_bomb();
    }
}
```

字符编码的高四位不会参与查表, 所以不同字符可能对应同一下标。求答案分两步: 先挑出六个表下标使和为 39, 再给每个下标选择合法字母或数字。aaachh 对应下标 1 1 1 3 8 8; 每个字符的贡献依次是 10 10 10 1 4 4。

9. Phase 6: 链表重排

输入是 1-6 的一个排列。范围检查把输入减 1, 再做无符号 ja, 同时排除小于 1 和大于 6 的值; 双重循环排除重复值。

随后对每个数做 $x \mapsto 7 - x$ 。转换后的数表示原链表中的第几个节点: 从 node1 出发, 反复读取节点 +8 处的下一节点指针。选出的六个指针存到栈中, 按输入顺序重新连接, 最后一项的 next 置 0。

节点布局: 偏移 0 是 4 字节值, 偏移 4 是 4 字节编号, 偏移 8 是 8 字节 next 指针。x/4dw &node1 显示的后两个整数只是同一指针的两半; 查看 next 应用 x/gx ((char *)&node1+8)。node6 地址不连续, 必须沿指针或按符号读取。

降序位置	原节点	节点值	输入: 7-原位置
1	5	917	2
2	1	776	6
3	6	630	1
4	3	615	4
5	2	541	5
6	4	201	3

最终检查为 `cmp %eax, (%rbx)` 后 `jge`, 其中 `%eax` 是下一节点值。因此要求“当前值大于等于下一值”, 即有符号非递增排列。排序后原节点顺序为 5 1 6 3 2 4, 转换回实际输入为:

2 6 1 4 5 3

9.1. 第一段: 范围、重复与数值变换

这一关预留 `0x50=80` 字节局部空间。栈的前 48 字节存六个 8 字节节点指针, 后面的 24 字节存六个 4 字节输入整数:

```
RSP + 0x00 ... RSP + 0x2f   selected[6]: pointers, 6 * 8 bytes
RSP + 0x30 ... RSP + 0x47   numbers[6]: integers, 6 * 4 bytes
RSP + 0x48 ... RSP + 0x4f   remaining local space
```

外层 `%ebp=i`, 内层 `%ebx=j`, 内层从 `i+1` 起检查, 避免把元素与自身比较。`%r12d` 保存外层下一下标, 让内层结束后可更新外层。

```
17d0: mov 0x30(%rsp,%rax,4),%eax
17d4: sub $1,%eax
17d7: cmp $5,%eax
17da: ja failure                # Require unsigned(x - 1) <= 5.
17dc: lea 1(%rbp),%r12d         # Save i + 1.
17e0: mov %r12d,%ebx           # Start j at i + 1.
17b4: mov 0x30(%rsp,%rdx,4),%edi
17b8: cmp %edi,0x30(%rsp,%rax,4)
17bc: jne next_j               # Unequal inputs are allowed.
17be: call explode_bomb
```

此代码块按“外层准备、内层检查”分组展示, 实际执行在两段地址间跳转。输入 1 时减 1 得 0, 输入 6 时得 5, 均允许; 输入 0 时得 `0xffffffff`, 无符号值非常大, `ja` 成立。减法应按机器的 32 位模运算理解, 等价表达可写为 `(unsigned)x-1u`, 避免 C 的有符号溢出问题。

```
17ef: mov $7,%edx
17f4: sub 0x30(%rsp,%rcx,4),%edx
17f8: mov %edx,0x30(%rsp,%rcx,4)
```

变换会原地覆盖输入整数, 检查栈内数组时要区分变换前后: 最终输入 2 6 1 4 5 3 会变为 5 1 6 3 2 4。

9.2. 第二段: 按链表位置取节点

```
1827: mov $1,%eax              # Position starts at 1.
182c: lea ...(%rip),%rdx       # Start at node1.
1833: jmp 1812
180b: mov 0x8(%rdx),%rdx       # Follow the next pointer.
180f: add $1,%eax              # Advance the position.
1812: movslq %esi,%rcx         # rcx = input index i.
1815: cmp %eax,0x30(%rsp,%rcx,4)
1819: jg 180b                  # Continue while target > position.
181b: mov %rdx, (%rsp,%rcx,8)  # Save selected[i].
```

如果目标为 1，立即保存 `node1`，无需移动；目标为 5，则沿 `next` 移动四次。最后的 ,8 对应指针数组，前面的 ,4 对应整数数组。这里是遍历链表顺序，程序没有读取节点 +4 的编号字段来搜索；编号与原位置一致只是本次数据的性质。

```
struct Node {
    int value;           // Value used by the order check.
    int number;          // Node identifier, not the selection key.
    struct Node *next;   // Address of the next node.
};

// Select nodes by their one-based positions in the original list.
for (int i = 0; i < 6; ++i) {
    struct Node *p = &node1;
    for (int position = 1; position < numbers[i]; ++position) {
        p = p->next;
    }
    selected[i] = p;
}
```

这一步只保存指针，还没有重写 `next`，所以六次选择都遍历原链表。若先重连再选，会改变后续遍历结果；原程序的阶段顺序保证了选择的一致性。

9.3. 第三段：重新连接并检查降序

```
1835: mov (%rsp),%rbx      # rbx = selected[0], the new head.
1839: mov %rbx,%rcx         # rcx = current node.
1846: mov (%rsp,%rdx,8),%rdx # rdx = selected[i].
184a: mov %rdx,0x8(%rcx)     # current->next = selected[i].
1851: mov %rdx,%rcx          # Advance current.
1859: movq $0,0x8(%rcx)      # Terminate the list.
1874: mov 0x8(%rbx),%rax     # rax = current->next.
1878: mov (%rax),%eax        # eax = next->value.
187a: cmp %eax,(%rbx)        # Test current->value - next->value.
187c: jge next_pair          # Require current >= next.
187e: call explode_bomb
```

最后只有五次相邻比较，因为六个节点只有五对相邻关系。要求的是非递增，不是严格递减；本次值各不相同，所以实际表现为严格降序。

9.4. 全关等价代码

前半部分恢复输入检查和数字变换：

```

/** Check and reorder the six nodes for phase 6. */
void phase_6(const char *input) {
    int numbers[6];
    struct Node *selected[6];
    read_six_numbers(input, numbers);

    // Check that the inputs form a permutation of 1 through 6.
    for (int i = 0; i < 6; ++i) {
        if ((unsigned)numbers[i] - 1u > 5u) {
            explode_bomb();
        }
        for (int j = i + 1; j < 6; ++j) {
            if (numbers[i] == numbers[j]) {
                explode_bomb();
            }
        }
    }
    for (int i = 0; i < 6; ++i) {
        numbers[i] = 7 - numbers[i];
    }
}

```

后半部分继续同一个函数；两个代码块之间没有额外操作：

```

// Select all nodes before changing any next pointer.
for (int i = 0; i < 6; ++i) {
    struct Node *p = &node1;
    for (int position = 1; position < numbers[i]; ++position) {
        p = p->next;
    }
    selected[i] = p;
}

// Build the new list and check each adjacent pair.
for (int i = 0; i < 5; ++i) {
    selected[i]->next = selected[i + 1];
}
selected[5]->next = NULL;
struct Node *p = selected[0];
for (int i = 0; i < 5; ++i) {
    if (p->value < p->next->value) {
        explode_bomb();
    }
    p = p->next;
}
}

```

验证时可在 phase_6+18 读取原输入，在 phase_6+126 读取变换后的数字，在 phase_6+175 查看六个指针。后一个位置尚未重连链表。

```

b *(phase_6+18)
b *(phase_6+126)
b *(phase_6+175)
# At each breakpoint, confirm the current instruction first.
x/i $pc
x/6dw $rsp+0x30
# At phase_6+175, inspect the completed pointer array.
x/6gx $rsp

```

六个指针的预期顺序为 `node5,node1,node6,node3,node2,node4`。实际地址依本次加载而定；重跑程序会重新加载原节点数据，不应把已重连的内存当作原链表继续推导。

10. 隐藏关入口：两道口令

`phase_defused` 先完成当前关的服务器校验；只有状态为 1 且 `num_input_strings=6` 时，才检查隐藏入口。必须让 `abracadabra()` 与 `alohomora()` 都返回非零，程序才会正常调用 `secret_phase`。不要通过跳转或直接调用来代替入口条件。

10.1. 第二行：abracadabra

它对 `input_strings+120` 执行 `sscanf`。每条输入槽占 120 字节，所以这是第二行。格式与目标分别为：

```
%d %d %d %d %d %d %s
NothingThatHasMeaningIsEasy...
```

必须成功转换 7 项，并使最后字符串完全相同。因此第二行改为：

```
0 1 1 2 3 5 NothingThatHasMeaningIsEasy...
```

第二关本身只读取前六个整数，追加口令不改变它的数列检查。

关键判断可整理为以下语义。`input_strings` 是输入槽数组，这里的二维写法用于说明每槽 120 字节的布局。

```
/** Return 1 if input line 2 contains the required extra word. */
int abracadabra(void) {
    int a[6];
    char word[136];
    int count = sscanf(input_strings[1],
                       "%d %d %d %d %d %d %s",
                       &a[0], &a[1], &a[2],
                       &a[3], &a[4], &a[5], word);

    if (count != 7) {
        return 0;
    }
    return strings_not_equal(
        word, "NothingThatHasMeaningIsEasy...") == 0;
}
```

该函数不重新检验六个数的递推关系；数字的正确性已由第二关与成功校验处理。它只利用同一行的格式取出末尾口令。原始 `%s` 未带宽度限制，上面的数组大小依据栈空间表达；此处是对给定短口令的逆向说明，不是推荐通用的字符串读取写法。

汇编在调用 `sscanf` 前多次移动栈指针，所以某个 `lea 0x28(%rsp)` 的绝对栈地址要结合当时 `%rsp` 理解。调用后 `add $0x20,%rsp` 撤销参数空间，`lea 0x20(%rsp),%rdi` 才是同一口令缓冲区在恢复后的偏移。偏移数字不同，不一定表示不同对象。

10.2. 第四行：alohomora

它对 `input_strings+360`（第四行）执行 `sscanf`，格式为 `%d %d %s`。成功转换 3 项后，把追加字符串中每个字符的编码加 2，再与目标比较：

```
000Gcu{FqgupvGpvgvt3pvqItqypWrNkhg0
```

因此实际输入应将目标的每个字符编码减 2。例如 `0` 变为句点，`G` 变为 `E`，`3` 变为数字 `1`。得到：

```
24 2 ...EasyDoesntEnter1ntoGrownUpLife.
```

两道口令中都是数字 1，不是小写字母 `i`；第二行结尾为三个句点，第四行开头三个句点、结尾一个句点。

10.3. 字符加 2 的循环与成功处理控制流

```

1537: add $2,%eax
153a: mov %al,(%rdx)          # Write one changed byte.
153c: add $1,%rdx
1540: movzbl (%rdx),%eax
1543: test %al,%al
1545: jne 1537
1547: lea 0x10(%rsp),%rdi
154c: lea ...(%rip),%rsi      # Address of the encoded target.
1553: call strings_not_equal
1558: test %eax,%eax
155a: je success
1563: lea 0x10(%rsp),%rdx      # Entry after three parsed fields.
1568: jmp 1540

```

入口在最后两条指令，先跳到 1540 检查结束符，非零字符才加 2 并写回；不改变结尾的 `\0`，所以字符串长度不变。改变的是解析到栈上的口令副本，不是答案文件，也不是全局输入行。

```

/** Return 1 if line 4 contains the encoded-target preimage. */
int alohomora(void) {
    int a, b;
    char word[136];
    if (sscanf(input_strings[3], "%d %d %s", &a, &b, word) != 3) {
        return 0;
    }
    for (char *p = word; *p != '\0'; ++p) {
        *p = (char)((unsigned char)*p + 2);
    }
    const char *target = "000Gcu{FqgupvGpvg3tpvqItqypWrNkhg0";
    return strings_not_equal(word, target) == 0;
}

```

`phase_defused` 的关键条件可恢复为以下控制流。非隐藏分支中的提示输出省略，正常成功消息发送仍保留：

```

/** Report success and enter the secret phase when both gates pass. */
void phase_defused(int *status) {
    *status = 0;
    send_msg(1, status);
    if (*status != 1) {
        exit(8);
    }
    if (num_input_strings == 6 && abracadabra() != 0) {
        if (alohomora() != 0) {
            secret_phase();
        }
    }
}

```

不同检查函数的返回值约定并不相同：`strings_not_equal` 是 0 表示相等；两道入口函数是 1 表示通过；隐藏关同步检查又是 0 表示通过。要根据调用者的 `je/jne` 恢复含义，不能统一认为“非零就成功”。隐藏关读取第七行后 `num_input_strings=7`，再次调用成功处理不会重新满足 `==6`，所以不会再次进入隐藏关。

11. Secret phase: 有限状态机的同步序列

11.1. 字符与长度检查

隐藏关再调用一次 `read_line`。扫描先判断是否到字符串结束符，再判断下标是否大于 16。因此长度为 17 时，下标 17 恰为结束符，仍然通过；最大长度是 17，不是 16。

`emulate_fsm` 将字符减去 '0'，用无符号检查限制结果为 0 或 1，所以只能输入二进制字符。状态保存在 `%ebp`，更新规则为：

$$q' = T[7d + q], \quad d \in \{0, 1\}, \quad q \in \{0, 1, 2, 3, 4, 5, 6\}.$$

`lea(, %rax, 8), %rdx` 后再 `sub %rax, %rdx`，实际上计算 $7d$ ；表项乘 4 是因为每项为 `int`。

11.2. 长度边界的准确恢复

```
1935: mov $0,%ebx           # i = 0.
193a: jmp 193f
193c: add $1,%ebx           # i++.
193f: movslq %ebx,%rax
1942: cmpb $0,(%rbp,%rax)   # Test input[i] first.
1947: je 1955               # Accept the end of the string.
1949: cmp $0x10,%ebx        # Compare i with 16.
194c: jle 193c              # Advance only while i <= 16.
194e: call explode_bomb
```

长度 17 时，下标 0–16 仍有字符，下标 17 为结束符，直接跳到 1955；长度 18 时，下标 17 仍有字符，随后发现 $17 > 16$ ，才失败。边界由“先测结束符、再测下标”的顺序决定。

```
/** Check the secret input length, symbols, and synchronization. */
void secret_phase(void) {
    const char *input = read_line();
    int i = 0;
    while (input[i] != '\0') {
        if (i > 16) {
            explode_bomb();
        }
        ++i;
    }
    if (check_synchronizing_sequence(input) != 0) {
        explode_bomb();
    }
    int status;
    phase_defused(&status);
}
```

空串虽满足长度条件，但从七个不同起点执行空串会得到七个不同结果，不能通过同步检查。字符限制由 `emulate_fsm` 内部完成。

当前状态	0	1	2	3	4	5	6
输入 0	5	1	6	4	2	3	0
输入 1	1	6	2	3	4	5	1

11.3. 同步条件与求解

`check_synchronizing_sequence` 先从状态 0 执行字符串，保存最终结果；随后从状态 1–6 分别执行同一字符串。七次结果全部相同则返回 0，否则返回 -1。

11.3.1. 状态机与检查函数的等价代码

```

/** Return the final state after reading a binary word. */
int emulate_fsm(int state, const char *input) {
    const int table[14] = {
        5, 1, 6, 4, 2, 3, 0,
        1, 6, 2, 3, 4, 5, 1
    };
    while (*input != '\0') {
        unsigned digit = (unsigned char)*input - (unsigned)'0';
        if (digit > 1u) {
            explode_bomb();
        }
        state = table[7 * digit + state];
        ++input;
    }
    return state;
}

```

表的第一行对应字符 0，第二行对应字符 1；当前状态选择列。输入字符 1、状态 6 时读取下标 $7+6=13$ ，值为 1。状态 0 遇到字符 1 也到达 1，所以第一步会让起始状态 0 和 6 合并。

```

/** Return 0 if all seven start states reach the same end state. */
int check_synchronizing_sequence(const char *input) {
    int expected = emulate_fsm(0, input);
    for (int start = 1; start <= 6; ++start) {
        if (emulate_fsm(start, input) != expected) {
            return -1;
        }
    }
    return 0;
}

```

原汇编中 `%rbp` 保存字符串地址，`%r12d` 保存从状态 0 出发的目标结果，`%ebx` 是当前起始状态。每次调用前重新把字符串首地址放入 `%rsi`，意味着七次都从字符串开头读取，不是接着上一次的末尾读取。

求解可从集合 $S = \{0, 1, 2, 3, 4, 5, 6\}$ 出发，读取字符时令 $S' = \{T[7d + q] : q \in S\}$ 。相同的新状态只保留一次。对这些状态集合做广度优先搜索，第一次到达单元素集合便得到最短同步序列；这只计算已读取的转换表，不反复向 bomb 或服务器试答案。

11.3.2. 离线求解与独立验证

下面的独立 Python 计算只使用转换表。`frozenset` 表示不可变集合，可放入 `seen` 去重；队列保证先探索较短字符串。总共最多有 $2^7 - 1 = 127$ 个非空状态集合，远少于逐个生成所有长度不超过 17 的字符串。

```

from collections import deque

# Each row maps the seven current states for one input digit.
TRANSITIONS = (
    (5, 1, 6, 4, 2, 3, 0),
    (1, 6, 2, 3, 4, 5, 1),
)

def shortest_sync_word():
    """Return a shortest binary word that merges all seven states.

    Returns:
        The word, or None if the transition graph has no such word.
    """
    initial = frozenset(range(7))
    queue = deque([(initial, '')])
    seen = {initial}
    while queue:
        states, word = queue.popleft()
        if len(states) == 1:
            return word
        for digit in (0, 1):
            next_states = frozenset(
                TRANSITIONS[digit][state] for state in states
            )
            if next_states not in seen:
                seen.add(next_states)
                queue.append((next_states, word + str(digit)))
    return None

```

到达同一状态集合的两个前缀，其未来可能到达的集合完全相同；较长前缀无需重复探索。集合只能保持大小或缩小，不能增加大小，但可能在相同大小的不同集合之间移动，因此仅按“立即减少状态数”贪心选择字符并不可靠。

再单独模拟每一个起始状态，核对输出长度和七个最终状态：

```

def final_state(start, word):
    """Return the end state for a start state and a binary word."""
    state = start
    for character in word:
        state = TRANSITIONS[int(character)][state]
    return state

word = shortest_sync_word()
print(word)
print(len(word))
print([final_state(start, word) for start in range(7)])
# 10101010100000101
# 17
# [1, 1, 1, 1, 1, 1, 1]

```

这一步分别验证“找到最短序列”和“七个起点确实到达同一个终点”。运行的是离线状态机模型，不是 bomb 程序。结果为：

```
10101010100000101
```

共 17 个字符。按前缀推进，可能状态集合如下，最后收敛到状态 1：

已读字符数	输入前缀	可能状态集合
0	空串	0,1,2,3,4,5,6
1	1	1,2,3,4,5,6
3	101	1,2,3,4,6
5	10101	1,2,4,6
7	1010101	1,2,6
9	101010101	1,6
14	10101010100000	1,2
15	101010101000001	2,6
16	1010101010000010	0,6
17	10101010100000101	1

12. 最终答案、验证与提交

将 `psol.txt` 保存为以下七行，最后一行也保留换行。口令是追加在原行中的字段，不是另外插入的输入行。

```
Love is a program that science and logic can never explain.
0 1 1 2 3 5 NothingThatHasMeaningIsEasy...
0 42
24 2 ...EasyDoesntEnter1ntoGrownUpLife.
aaachh
2 6 1 4 5 3
10101010100000101
```

在 xLab 的 `bomb406` 目录中用 GDB 重新运行，普通断点处用 `c` 继续，让 `phase_defused` 实际执行；从旧停点继续不能替代一次读取已保存文件的完整运行。若此前对寄存器或栈做过临时修改，重新 `run` 可避免把临时状态误当作答案验证。

课程随附 `bomblab.pdf` 的 Handin 说明：**无需显式提交文件，程序会自动向 Autolab 上报进度**。通过后到 Autolab 的 Bomblab → View scoreboard 核对记录。`psol.txt` 是本地输入与复现文件，并不等同于平台已经接受答案。本次历史配置保留成功上报，因此无需为了提交而恢复二进制；这也不证明本次聊天中所有答案已经实际被服务器接受。

课程说明给出六关共 100 分：前四关各 15 分，后两关各 20 分；隐藏关不计分。这里复述的是随附说明，若课程后续发布更正，应以新要求为准。

12.1. 排错清单

- **停点不对**：先 `x/i $pc` 和 `bt`，再决定寄存器和栈偏移的含义。
- **读取失败**：调用 `sscanf` 前查看 `$rdi` 输入与 `$rsi` 格式，调用后查看 `$eax` 转换数量。
- **答案没有更新**：检查文件是否保存、GDB 参数是否为 `psol.txt`、行序是否正确，再 `run`。
- **不断停在旧关卡**：先查询断点编号，再禁用已完成关卡的临时断点；不要把安全断点一并禁用。
- **程序退出**：The program is not being run. 后不能 `c`，需重新 `run`；仍可查看已加载程序的反汇编。
- **隐藏关未触发**：确认已读六行、成功校验状态为 1、第二与第四行口令原样附加，且没有跳过入口函数。

12.2. 关键断点索引 (bomb406)

位置	相对断点	检查内容
第一关比较前	phase_1+11	\$rdi 与 \$rsi 字符串。
第二关读取后	phase_2+13	x/6dw \$rsp。
第三关解析前	phase_3+26	格式、输入与两个目标指针。
第三关最终比较	phase_3+121	\$eax 与 \$rsp+8。
第四关解析前	phase_4+26	确认 %d %d 与实际输入。
第四关最终比较	phase_4+69	\$eax=24, 输入 24 2。
第六关顺序检查	phase_6+244	当前节点值与下一节点值。
隐藏关检查返回后	secret_phase+54	\$eax=0 表示同步成功。

复习时优先把握四件事：从参数寄存器确定输入位置，从条件跳转恢复约束，从循环与递归恢复计算关系，从内存表与指针恢复数据结构。动态观察应验证这些关系，而不是代替对控制流的理解。